



使いすぎないブロックチェーン

— Symbolで考える署名ドリブン設計 —

さだかね / nemnesia
v1.0 / 2026-02-15

はじめに

本書は、ブロックチェーンの使い方を網羅的に解説する本ではありません。また、特定のチェーンやプロダクトを推奨したいわけでもありません。

ただ、例として扱う多くの題材は Symbol を前提にしています。ここは最初に正直に書いておきます。

Symbol について話していると、ときどきこう問われます。

「Ethereum と同じことができるのか」

「もっと派手なことはできないのか」

結論から言えば、同じことはできません。Symbol は、チェーン上で汎用ロジックを動かしてアプリの状態遷移を強制するタイプではないからです。

けれど、それを「不足」と捉える必要はありません。Symbol は意図的に、「決定しない」「実行しない」方向へ寄せています。代わりに、誰が・いつ・何に署名したかという事実を、改ざんできない形で残すことに集中しているのです。

ブロックチェーンは「何でも載せられる」ようになりました。その結果、状態管理や業務ロジック、実行結果、権限管理までを、すべてトランザクションで片づけようとする設計も珍しくありません。確かに強い。けれど同時に、実装や運用は重くなり、変更も難しくなります。

では、ブロックチェーンに本当に任せたいものは何でしょうか。本書では、誰が、いつ、何を承認したのかという事実を中心に、システムを組み立てる視点をとります。事実だけが確定していれば十分で、実行や状態の管理は必ずしもチェーン上で行う必要はありません。

Symbol は、こうした判断を“無理なく”成立させやすい基盤です。だから本書では結果として Symbol を使っています。制約ではなく、設計との相性として選びました。

本書は、ブロックチェーンを「どう使うか」よりも、「どこで使わないか」を考えるための本です。その判断を自分の設計に引き寄せ、考え直すきっかけになればと思います。

第1章：ブロックチェーンを“使いすぎる”と何が起きるか

1.1 改ざんできない、という強すぎる魅力

ブロックチェーンに触れたとき、
多くの人が最初に魅力として感じるのは
「改ざんできない」という性質です。

データは一度書き込まれ、
誰かの都合で書き換えられることはありません。
履歴はすべて公開され、
後から検証できます。

この特徴は、とても分かりやすく、
そしてとても強いものです。

だからこそ、
「重要なものは全部ここに置くべきだ」
という発想が生まれます。

状態、履歴、判断の結果。
できるだけ多くの情報をチェーンに載せるほど、
システムは安全になるように思えます。

特に、
従来のデータベース運用で苦労した経験がある人ほど、
この発想に強く引き寄せられます。

1.2 「決定」を人から切り離したくなる

もう一つの理由は、
誰が決めたのかを考えたくない
という気持ちです。

人が関わると、責任が生まれます。
責任が生まれると、
議論や調整が必要になります。

ときには、
不正や恣意性も入り込みます。

そこで、
「チェーンに決めさせる」
という設計が魅力的に見えてきます。

スマートコントラクトにルールを書いてしまえば、
あとは自動で処理される。
誰かが判断した、という事実は表に出ません。

この構造は、
公平で中立な仕組みに見えます。

しかし、
ここで一つだけ考えてみたい点があります。

ブロックチェーンは、
本当に「決定を引き受ける存在」なのでしょうか。

1.3 載せすぎた結果、何が起きるか

ブロックチェーンは、
一度載せたものを消せません。

仕様を間違えたとしても、
「なかったこと」にはできません。
後から修正する場合も、

過去の状態を背負い続けることになります。

その結果として、

- トランザクションが肥大化する
- 状態管理が複雑になる
- 設計変更のコストが跳ね上がる

といった問題が、
少しずつ蓄積していきます。

安全性を求めたはずなのに、
運用の自由度は下がり、
作る側も使う側も疲れていきます。

それでもなお、
「全部載せ」の設計から抜け出せない。

それは、
ブロックチェーンの問題ではありません。

私たちの設計の癖です。

1.4 本当に残したいものは何か

ここで、
少し視点を変えてみます。

ブロックチェーンが得意なのは、
データそのものを保存することではありません。

- 「いつ、誰が、何に同意したか」
- 「その行為が確かに行われたという事実」

そういった、
行為の証跡を残すことです。

すべてを載せる必要はありません。
しかし、
何も載せないわけでもありません。

この中間に、
別の設計の余地があります。

次の章では、
その余地を支える中心概念として
「署名」に注目していきます。

第2章：ブロックチェーンに本当に残したいもの

この章では、署名を「認証のための技術」から切り離し、**意思決定をどのようにシステムに残すか**という設計思想として再定義します。

パスワード中心の設計では見えなかったものが、署名を主役に据えた瞬間に浮かび上がります。

2.1 署名とは何か——正しさではなく、意思を固定するもの

多くのシステムにおいて、署名は「認証の裏側」に隠れた技術要素として扱われています。

- ログイン時に使われるもの
- 正しさを検証するためのもの
- 暗号として安全かどうか

ここまでが一般的な理解でしょう。

しかし、紙の契約書を思い出してみてください。
私たちは署名するとき、ハッシュ関数や鍵長を意識しません。

- 内容を読み
- 理解し
- 自分の名前を書く

署名が意味を持つのは、
そこに人の判断が介在しているからです。

署名とは「正しさの証明」ではなく、「意思決定の固定化」です

デジタル署名であっても、この本質は変わりません。

2.2 署名は対話である——チャレンジとレスポンス

意思決定を固定する以上、署名は一方的に検証されるだけの存在ではありません。
そこには必ず、**問いと応答**の関係が生まれます。

1. システムが問いを投げ
2. ユーザーが内容を確認し
3. 署名という形で応答する

この流れは、
単なる認証ではなく、**小さな対話**だと言えます。
重要なのは、署名される対象が意味を持つことです。

- 単なる乱数
- サーバ都合のトークン

ではなく、

- 「この操作を承認する」
- 「この条件に同意する」

といった、文脈を含んだメッセージである必要があります。

このように、人が判断できる単位までシステムを分解し、その判断を署名として残す設計は、従来の認証中心の考え方とは異なる視点を持っています。

2.3 承認と実行を分離する——署名を中心に据える

従来の Web システムでは、

- 認証すると
- 権限が付与され
- あとはサーバが実行する

という流れが一般的でした。

この設計では、

誰が

何を

どこまで理解して

承認したのかが、システムの奥に埋もれてしまいます。

署名を中心に据えた設計では、ここを意図的に分離します。

- 承認＝署名
- 実行＝別レイヤの処理

実行は失敗しても構いません。

再試行しても構いません。

別の主体が行っても問題ありません。

しかし、承認だけは改ざんできない形で残す

この分離によって、

- 監査性
- 再現性
- 責任分界

が設計段階で明確になります。

2.4 関係の逆転——攻撃点は分散できる

パスワード中心の設計では、
攻撃点はサーバ側に集中します。

- 認証情報を預かり
- セッションを管理し
- 実行権限を保持する

署名を中心に据えた設計では、この関係が逆転します。

- 鍵はユーザー側にあり
- サーバは署名を検証するだけ
- 実行権限は署名に紐づく

その結果として、
攻撃点は自然に分散されます。

これはセキュリティ技術の話というより、
責任と権限をどこに配置するかという設計の話です。

2.5 署名が「責任」を生む理由

署名が責任を生むのは、感情的な理由ではありません。
否認が困難だからです。

- パスワードは共有されたと言えます
- セッションは奪われたと言えます

しかし署名は、

- その鍵で
- その内容に
- その時点で

行われた事実を第三者が検証できます。

署名を中心に据えた設計とは、

行為ではなく、意思決定を第一級のデータとして扱う

という宣言でもあります。

2.6 技術スタックに依存しない思想

この章で述べた考え方は、特定の製品やチェーンに依存しません。

- ブロックチェーンでなくても構いません
- Web2 でも理論上は実現可能です

ただし現実には、

この考え方を自然に実装できる基盤は多くない

という問題に直面します。

- 署名が中心に据えられていない
- 承認ログが後付けになりがち
- 実行主体が強すぎる

ここで初めて、**基盤そのものの設計**が問題として浮かび上がります。

本書では以降、
このように**署名を起点として設計を組み立てる考え方を**、
便宜上「署名ドリブン設計」と呼びます。

次章では、
「署名が主役として扱われている基盤」という視点から、
具体例を一つ取り上げます。

それが **Symbol** です。

第3章：なぜ Symbol が自然にハマるのか

第1章では、ブロックチェーンの「改ざんできない」という魅力が、
状態や実行結果までを“全部載せ”したくなる設計の癖を生むことを見ました。
そして第2章では、署名を「認証の裏側」から切り離し、
意思決定を固定して残す方法として捉え直しました。

ここまでの流れを受けて、この章で扱う問いは一つです。

署名を主役に据えると、基盤の前提はどう変わるのか

署名ドリブン設計の考え方自体は、特定のチェーンや製品に依存しません。
理論上は Web2 でも成立します。
ただし現実には、**その思想を“無理なく”実装できる基盤**は多くありません。
多くの基盤が、「実行」や「状態」を主役として設計されているからです。

本章ではまず、
「署名が主役として扱われている」とは基盤設計として何を意味するのかを整理します。
その具体例として Symbol を取り上げます。

3.1 基盤は、何を「一番」に扱うか

すべての基盤設計には、
明示されていなくても、
暗黙の優先順位があります。

- 何が中心に置かれているのか
- 何が前提として扱われるのか
- 何が補助情報として付け足されるのか

この優先順位は、
後から設計思想を変えようとしても、
簡単には覆りません。

多くのシステムでは、
「実行」や「状態」が中心に据えられています。

- どの処理が実行されたか
- 状態がどう変化したか
- 現在の値が何であるか

署名は、
それらを支える付随情報として扱われがちです。

一方で、署名ドリブン設計では、
この優先順位が逆転します。

中心に置かれるのは、
処理の結果ではありません。

誰が、その内容を承認したか
という事実です。

何が起きたかよりも先に、
「それは誰の意思か」を確定させます。

署名は補助情報ではなく、
基盤が最初から扱うべき主役になります。

第2章で述べた「承認と実行を分離する」という設計判断は、
承認（署名）が“後付けのログ”ではなく、
基盤の中心として扱われているほど実装しやすくなります。

3.2 主体が署名によって定義される基盤

署名を主役に置く設計では、
主体の定義そのものが変わります。

名前や ID、
ロールや権限といった属性は、
主体そのものを表してはいません。

それらはすべて、
「主体をどう扱うか」という
運用上の都合にすぎません。

署名ドリブン設計において主体とは、
署名できる存在そのものです。

裏を返せば、
署名できないものは主体ではありません。

この割り切りは、
一見すると不親切に見えるかもしれません。

しかし、
「誰の意思か」を曖昧にしないためには、
非常に重要な前提です。

Symbol では、
この割り切りが最初からなされています。

アカウントとは本質的に公開鍵であり、
ネットワークが信頼するのは、
「どの公開鍵が署名したか」という一点だけです。

主体は宣言されるものではなく、署名によって示されます。

3.3 署名者が消えない設計

署名を主役にする設計では、「誰が関与したか」が必ず記録として残ることが前提になります。

これは単なる監査や履歴の話ではありません。人の意思を扱う以上、後から解釈が揺れない形で残る必要があります。

Symbol のトランザクションには、必ず署名者 (signer) が存在します。
しかもこれはオプションではなく、プロトコルの前提条件です。

- ・ マルチシグでも、内部トランザクションごとに署名者が存在する
- ・ アグリゲートでも、誰がどこまで承認したかが分離される
- ・ 「実行されたが、誰が承認したか分からない」という状態がない

この性質は、人の行為を起点とする用途で特に強く効いてきます。

たとえばログインや認可のように、重要なのは「誰がログインしたか」ではなく、「誰が、その時点で、その内容に署名したか」という一点です。

Symbol はこの問いに対し、追加の概念や仕組みを持ち込むことなく、プロトコルだけで自然に答えを返してくれます。

3.4 合意された順序と時点

承認は一回限りで、履歴としては順序も持ちます。署名ログインは、その分かりやすい例です。

Symbol では、トランザクションがブロックに取り込まれた時点で、ブロック高とネットワーク時刻が付与されます。ここで確定するのは、端末で「いつ署名ボタンを押したか」ではなく、承認として扱える時点とチェーン上の順序です。

確定する事実は、次の三つに絞れます。

- 承認として扱える時点（ブロックに取り込まれた時点）
- チェーン上の順序（ブロック高と取り込み順）
- 端末の署名時刻やオフチェーンの状態は別物であること（だから分離できる）

署名操作の時刻は端末や人に依存します。それでも、承認として「いつ成立したか」「どちらが先か」はチェーン上で揺れません。ログイン履歴や認可履歴を後から辻褃合わせで管理しなくてもよくなるのは、この一点が効いています。

3.5 運用コストという制約

思想が美しくても、運用できなければ意味がありません。

署名ログインは、

- 頻度が高い
- 金銭的価値を直接生まない
- 失敗が許されにくい

という性質を持ちます。

Symbol はこの点で、比較的現実的です。

- トランザクション手数料が低く安定している
- 複雑なスマートコントラクトを必要としない
- ノード運用や API 設計が比較的素直

特に重要なのは、

「署名＝高コスト」という感覚が生まれにくいことです。
これは UX にも思想にも直結します。

3.6 なぜ Symbol は無理をしないのか

多くのスマートコントラクト系チェーンは、「実行」を中心に設計されています。

- 状態遷移をどう記述するか
- ロジックをどこまでオンチェーンに載せるか
- ガス計算と最適化

一方、Symbol は明確に違います。

- 署名を中心に置く
- 実行よりも承認を重視する
- ロジックは外に逃がし、証跡だけを残す

これは優劣ではなく、思想の違いです。

署名ログインや DID 的な用途では、

「複雑な実行」よりも「誰が、いつ、何を承認したか」が重要になります。

そして、Symbol が向いている理由は派手さではありません。

- 余計な抽象化がない
- 主体の定義がブレない
- 署名が中心に据えられている

この「それだけ」が、承認と実行を分離したい設計では致命的に効いてきます。

次章では、こうした前提の上で、

最小構成で署名ログインをどう作るかを見ていきます。

第4章：最小構成で考える署名ログイン

第3章では、署名を「認証の裏側」ではなく、**誰の意思かを確定させる一次情報**として扱ってきました。

では、その考え方をログインに当てはめると、設計はどこまで削れるのでしょうか。

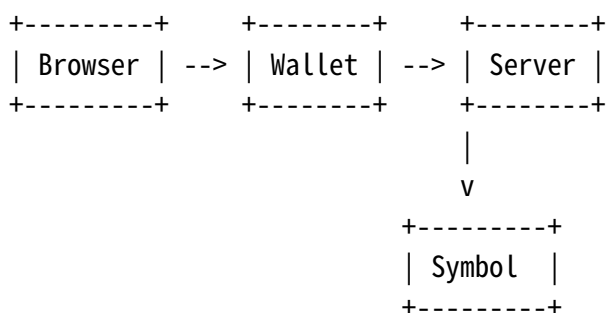
この章で扱う問いは、実装手順ではありません。

署名を主役にするなら、ログインはどこまでをチェーンに任せ、どこからを外に逃がすべきか

完成形を作る章ではなく、**成立条件を満たす最小単位**を整理する章です。

4.1 全体像 — まずは流れだけを見る

最小構成の署名ログインは、次の4者だけで成立します。



役割は単純です。

- Browser：操作の起点（信用しない）
- Wallet：署名を行う存在（主体）
- Server：検証とセッション管理
- Symbol：主体に関する事実の参照先

ここで重要なのは、チェーンは一度も「ログイン処理」をしないことです。

4.2 署名ログインの最小フロー

流れは驚くほど短くなります。

1. Server -> Browser：チャレンジを発行
2. Browser -> Wallet：署名を依頼
3. Wallet -> Browser：署名を返す
4. Browser -> Server：署名を送信
5. Server：検証してセッション発行

ここでログインを成立させているのは、「状態」でも「実行結果」でもありません。

その公開鍵に対応する秘密鍵が、この瞬間に操作された

という点だけです。

4.3 チェーンに乗せるもの／乗せないもの

署名ドリブン設計では、すべてをチェーンに乗せる必要はありません。

重要なのは、線引きです。

4.4 チェーンに乗せるもの

- 公開鍵と主体の結びつき
- 署名による承認の事実
- 後から争えない証跡
- （必要な場合のみ）承認として固定したい署名ログ

4.5 チェーンに乗せないもの

- セッション情報
- セッション履歴（ログイン回数など）
- 有効期限や失効管理

- UI や操作フロー

理由は単純です。

揮発してよいものを、消せない場所に置かない

4.6 Symbol の役割 — 実行しない基盤

この最小構成において、Symbol は「処理を動かす場所」ではありません。

- ログイン可否を判断しない
- セッションを管理しない
- アプリのロジックを持たない

代わりに、

主体（公開鍵）に関する事実を返す参照先

として使われます。

これにより、サーバは「本人性」を捏造できなくなります。

4.7 DID は目的ではなく、結果として現れる

この設計では、最初から DID を作ろうとはしていません。

- 署名を主役に置く
- 主体を公開鍵で定義する
- 参照先をチェーンに委ねる

その結果として、

- 鍵と主体が安定して結びつき
- 外部から検証可能になり
- DID 的な性質が現れる

という順序になります。

DID は設計目標ではなく、署名を正直に扱った結果です。

4.8 なぜ「嘘をつけない」のか

この構成は、チェーンを使っていない部分が多くあります。

それでも嘘をつけないのは、重要な一点だけが誤魔化せないからです。

- 誰が署名したか
- その鍵が本当に存在するか

ここだけは、チェーンが裏付けます。

チェーンに頼りすぎないが、嘘はつけない

これが、署名ログインの最小構成です。

次章では問いを反転させます。

では、何をオンチェーンに残すと価値が生まれるのか

第5章：何を残すと価値が生まれるのか

ブロックチェーンを使うとき、多くの人が最初に抱く期待はこうでしょう。
「これで、すべてを安全に保存できる」。

しかし、本当にそうでしょうか。

すべてをオンチェーンに残すことは、価値を生むのでしょうか。

本章では、この問いを掘り下げていきます。

結論から言えば、オンチェーンに残すべきなのは「必要最小限」だけです。
それは妥協ではなく、設計上の選択です。

5.1 オンチェーンは記録装置ではない

Symbol は、ストレージとしてのブロックチェーンではありません。
トランザクションやメタデータなど、確かにデータは記録できます。
しかも、ほかのチェーンと比べても容量には余裕があります。
しかし、それは「何でも載せる」ための場所ではありません。

なぜでしょうか。

第一に、コストがあるからです。

オンチェーンに書かれたデータは、ネットワーク全体で保持され続けます。
たとえ小さな文字列であっても、それは「全ノードに保存される重み」を持ちます。

第二に、意味の固定化が起こるからです。

一度書かれたデータは、文脈を失っても消えません。

仕様変更、運用変更、組織改編――

オフチェーンであれば「更新」で済むことが、オンチェーンでは永遠の過去になります。

だからこそ、Symbol を使うときに問うべきなのは、

「何を載せられるか」ではなく、

「何を載せるべきか」です。

5.2 意味を持たせすぎないメタデータ

Symbol のメタデータは強力です。

アカウント、モザイク、ネームスペースに対して、任意のキーと値を付与できます。

しかし、ここでもミニマリズムは重要です。

メタデータは「状態」ではなく、「ラベル」として使うのが適しています。

たとえば、次のような使い方です。

- このアカウントは「ログイン用」である
- この公開鍵は「承認者ロール」を持つ
- このネームスペースは「組織 ID」を表す

重要なのは、実体を載せないことです。

ユーザー情報そのもの、権限の詳細、履歴の全文――

それらはオフチェーンに置くべきです。

オンチェーンに残すのは、

「その解釈が正当であると検証できるための最小情報」だけで十分です。

5.3 承認だけを残すという選択

本書で扱ってきた署名ドリブン設計において、

オンチェーンが最も力を発揮するのが「証跡」です。

- 誰が
- いつ
- 何に対して署名したか

この三点は、Symbol では自然に残ります。

ログイン処理の中身――

チャレンジ文字列、セッション情報、ブラウザの状態――

それらはすべてオフチェーンで完結して構いません。

しかし、「この公開鍵の所有者が、この時点で承認した」という事実だけは、

オンチェーンに残す価値があります。

なぜならそれは、

後から検証できる「承認そのもの」だからです。

5.4 永続性を引き受ける設計

ブロックチェーンの特徴として、よく「消せない」ことが挙げられます。

しかし実際には、これは長所にも短所にもなります。

重要なのは、

消せないものを載せない設計を、最初から選ぶことです。

個人情報をご載せません。
意味が変わりうるデータを載せません。
将来、恥づかしくなる可能性のある情報を載せません。

オンチェーンに残すのは、
「残り続けても困らない事実」だけであるべきです。

署名したという事実。
承認が行われたという事実。
その順序と時刻。

それ以上は、欲張りません。

5.5 個人署名のその先へ

ここまでで見てきたのは、
個人の署名を起点として、

- 何をオンチェーンに残すのか
- 何をオフチェーンに逃がすのか

という設計判断でした。

しかし現実のシステムでは、
署名の主体は個人だけにとどまりません。
承認はやがて、複数人の合意となり、
「組織の意思」として扱われるようになります。

次章では、
マルチングを単なる複雑な仕組みとしてではなく、
署名が個人から組織へと拡張される自然な形として捉え直します。

個人の最小署名が、
どのようにして組織の合意へと接続されるのか。
その設計思想を見ていきましょう。

第6章：合意を署名で残す

ブロックチェーンにおける「署名」は、これまで主に個人の意思表示として語られてきました。
秘密鍵を持つ個人が署名し、その責任を引き受ける。
これは暗号技術としても、思想としても非常に明快です。

しかし、現実の社会や業務の多くは、個人だけでは完結しません。
稟議、承認、合意、決裁――
そこには必ず、組織としての意思決定があります。

本章では、Symbol のマルチングが単なるセキュリティ機能ではなく、
「組織が署名する」ための仕組みとして自然に成立していることを見ていきます。

6.1 個人署名と組織署名

個人署名はシンプルです。

「この人が承認した」という事実だけが分かれば十分です。

一方で、組織の意思決定はそう単純ではありません。

- 誰が提案したのか
- 誰が賛成したのか
- 誰が署名しなかったのか
- 合意が成立したのはいつか

これらはすべて、意思決定の一部です。

多くのシステムでは、

こうした情報はワークフローエンジンやデータベースの中に閉じ込められます。

最終的に「承認済み」という状態だけが残り、

途中の判断や異論はログの奥に埋もれていきます。

Symbol のマルチシグは、これをまったく違う形で扱います。

6.2 承認フローとしてのマルチシグ

Symbol のマルチシグは、表面的には「複数人で署名する仕組み」に見えます。けれど実態は、承認フローそのものです。提案が作られ、署名が集まり、条件を満たしたときだけ成立する。ここまですべてをプロトコルの枠内で扱えます。

- 提案はトランザクションとして作られる
- 各署名者は署名するか、しないかを選ぶ
- 必要な署名が揃ったときだけに成立する

重要なのは、確定した結果だけがオンチェーンに固定されることです。合意が成立すれば、誰が署名したかはチェーン上で確認できます。一方、成立条件を満たさなかったものは、確定ログとしては残りません。

「反対したこと自体も固定したい」なら、その意思を別の署名（メッセージ署名など）として残す設計もできます。全部を一つの仕組みに背負わせない。ここでも同じ方針が効きます。

また、署名が集まる順番は本質ではありません。大事なのは「誰が署名したか」と「必要数が揃ったか」であって、早い遅いではないからです。

6.3 マルチレベルで成立する合意

Symbol のマルチシグは、

マルチレベル構造を持つことができます。

- チーム単位の合意

- 部署単位の合意
- 組織全体としての最終合意

これらを、無理なく階層構造として表現できます。

複雑なワークフローエンジンを用意しなくても、
プロトコルレベルで
「どの範囲の合意が成立したか」だけが明確に残ります。

ここで扱われているのは、
役職や肩書きではありません。
誰かを識別するための ID でもありません。

残っているのは、
条件を満たした署名が集まり、
合意が成立したという事実だけです。

組織の意思決定を成立させるために、
恒久的な ID や属性情報は必須ではありません。

署名が集まり、条件を満たした。
それだけで十分なのです。

6.4 合意に必要な情報とは何か

ここまで見てきたように、Symbol のマルチシグは「複数人で安全に署名する仕組み」ではありません。

それは、組織としての意思決定を成立させ、その責任を署名として残すための仕組みです。

誰が賛成したか。誰が署名しなかったか。どの条件が満たされたときに、合意とみなされたか。

それらはすべて、役職や属性、恒久的な ID に依存せず、署名という事実だけで表現されます。

組織の意思決定に必要なのは、「誰であるか」を説明する情報ではありません。必要なのは、「誰がその判断に署名したか」が、後から検証できることです。

Symbol のマルチシグは、この一点にだけ集中することで、現実の組織的な合意を、過不足なくチェーン上に固定しています。

ここまでで、意思決定は「署名」で固定できることが見えてきました。では次に残る問いは、その署名者をどう扱うかです。言い換えれば、「誰」をどう設計に組み込むか、という問題になります。

第7章：ID を作らないという設計

7.1 分散 ID という出発点

では、その「誰」を、私たちはどのように扱えばよいのでしょうか。

すべてを自前で抱え込む必要があるのでしょうか。

ここで DID (Decentralized Identifier) という考え方が出てきます。

DID が目指している世界は、非常に美しいものです。

- 中央管理者に依存しない ID
- 自己主権的な鍵管理
- 検証可能な履歴
- グローバルに相互運用可能な識別子

理論上は、「誰が」「いつ」「どの鍵で」何を承認したかを、どこでも検証できる世界です。

この理想像自体を否定する必要はありません。むしろ、方向性としては正しいと言えます。

問題は、この理想をそのまま実装しようとしたときの現実にあります。

7.2 理想は、ときに重たい

DID を「ちゃんと」実装しようとする、急に構成要素が増えます。

- DID Document
- Verification Method
- Service Endpoint
- 鍵の失効・更新
- 履歴管理
- オフチェーン／オンチェーンの切り分け

これらは一つ一つが正しい設計要素ですが、組み合わせた瞬間に運用が重くなります。

特に問題になるのは、

「今、この署名は有効なのか？」

という問いに答えるために、複数の要素を横断的に確認しなければならない点です。

実装者にとっては学習コストが高く、利用者にとっては「よくわからない仕組み」になりがちです。

結果として、DID は「正しいが使われない技術」になりやすくなります。

7.3 集中しやすい Resolver という現実

DID の中核にあるのが Resolver です。

- DID 文字列を受け取り
- 対応する DID Document を取得し
- 検証に必要な情報を返す

理屈はシンプルですが、ここで重要な問題が生じます。

Resolverは誰が運用するのでしょうか？ Resolverは常に正しい情報を返すのでしょうか？ Resolverが落ちたら検証はどうなるのでしょうか？

「分散ID」を名乗りながら、検証の入り口が事実上の集中ポイントになるケースも多いです。

一方、Symbolではこうした問題が起きにくい。

- 公開鍵はアカウントそのもの
- 署名者はトランザクションに必ず残る
- 検証に必要な情報はチェーン上に揃っている

Resolverを挟まなくても、ブロックチェーン自体が検証基盤になります。

7.4 鍵ローテーションという現実

DID議論で必ず出てくるのが「鍵ローテーション」です。

- 鍵が漏洩したらどうする？
- 定期的に鍵を交換したい
- 過去の署名はどう扱う？

これは正しい問題提起ですが、現実にはこうした問いは用途依存です。

- ログイン用途なのか
- 承認ログなのか
- 資産操作なのか

すべてのユースケースでフルスペックの鍵ローテーション機構が必要なわけではありません。

Symbolでは、

- アカウントの公開鍵＝主体
- マルチシグによる段階的移行
- 用途ごとに鍵を分離する設計
- メタデータによるアカウントリンク設計

といった形で、過剰にならない現実的な選択肢を取ることができる。

すべてを一つの仕組みで解決しようとしない。それが、長く運用するための設計でもあります。

7.5 実用に必要な最小条件

ここで、問いを整理し直しましょう。

実用的な「分散的な承認・認証」に本当に必要なものは何でしょうか？

- 誰が署名したかが検証できる
- その署名が改ざんされていない
- いつ行われたかがわかる
- 後から第三者が確認できる

この4点が満たされていれば、多くの実用システムは成立します。

必ずしも、

- DID 文字列である必要はない
- DID Document を解決する必要もない
- Resolver を常設する必要もない

Symbol はこの最小条件を自然に満たす設計になっています。

7.6 「完全な DID を目指さない」という選択

本章で言いたいのは、「DID は不要だ」という主張ではありません。

むしろ、

DID にならなくても、DID 的な価値は実現できる

という選択肢が存在する、という整理です。

Symbol は、

- 署名が第一級市民
- 公開鍵が主体
- 検証がチェーンで完結する

という思想を持っています。

その結果、「DID を実装しようとして苦しくなる前に、もう十分なものが揃っている」という状態になっています。

理想に寄せることもできますが、無理に名前を合わせる必要はありません。

できることを、確実にやります。それ以外は、無理に抱え込みません。

第8章：署名ドリブン Symbol の使い道

ここまで、本書では一貫して

「署名を主役にする設計」

「ブロックチェーンを使いすぎない設計」

について語ってきました。

本章では、それらの思想が
具体的にどのような形で現実のシステムに落ちてくるのかを見ていきます。

重要なのは、
「Symbol だからできる派手なユースケース」ではありません。

- すでに存在している業務
- すでに存在している判断
- すでに存在している責任

それらを無理なく署名に置き換える。
それが署名ドリブン設計の実践です。

8.1 管理画面ログイン

最も分かりやすい適用先が、管理画面ログインです。

従来の管理画面は、以下のような構成を取ります。

- ID / パスワード
- 二要素認証
- IP 制限
- 操作ログ

これらはすべて
「誰が入ったか分からない」ことへの後付け対策です。

署名ログインでは、構造が逆転します。

- 署名できた人しか入れない
- 誰の鍵で署名したかが最初から確定している
- ログイン自体が意思表示として成立している

結果として、

- 認証
- 監査
- 責任所在

が、一つの署名に収束します。

ここでブロックチェーンは、「認証基盤」ではなく
署名のタイムスタンプ置き場として静かに機能します。

8.2 承認ワークフロー

次に自然に繋がるのが、承認フローです。

稟議、申請、承認、決裁。

これらは本質的に、

人が判断し、その痕跡を残したい

という行為です。

署名ドリブン設計では、

- 承認 = 署名
- 実行 = 別レイヤ

という分離がはっきりします。

重要なのは、「実行できた」よりも「承認した」ことです。

マルチシグを用いれば、

- 誰が賛成したか
- 誰が反対したか
- 誰がまだ署名していないか

が、そのまま履歴として残ります。

ここには DAO のような思想はありません。

現実の組織構造を、そのまま写すだけです。

8.3 なぜ「全部を載せない」のか

承認フローを設計していると、よくこう言われます。

「成立しなかったものが消えるのは問題ではないか」

ただ、ここで消えているのは判断そのものではなく、判断に付随する文章や業務文脈です。成立した判断は署名として残せますし、文章はオフチェーンに置けば更新も訂正もできます。両者を切り分けられない限り、ブロックチェーンは「巨大で、重くて、使いにくい台帳」になっていきます。

署名ドリブン設計は、記録を増やすための設計ではありません。残すものを絞り、運用を軽くするための設計です。

8.4 NFC / スマホ署名

署名は必ずしも「画面操作」を必要としません。

NFC やスマートフォンを用いれば、

- 端末をかざす
- 署名する
- 応答を返す

という、極めて物理的な操作に落とし込めます。

ここで重要なのは UX ではなく、責任の構造です。

- 触れた人
- 署名した鍵
- その時刻

これらが揃った瞬間、

行為としての署名が成立します。

「本人が操作したかどうか」を推測する必要はありません。

署名がすべてを語ります。

8.5 IoT / 物理鍵

IoT や物理鍵の世界では、

「認証」は常に悩みの種です。

- デバイス ID
- トークン
- API キー

どれも管理が難しく、漏洩時の影響が大きい。

署名ドリブン設計では、

- デバイス = 鍵を持つ存在
- 行為 = 署名

というシンプルな構造になります。

ドアを開ける、機械を動かす、設定を変える。

それらはすべて、

「この鍵が、この行為を許可した」

という署名に還元できます。

物理世界と論理世界をつなぐのは、

ID ではなく**署名という行為**です。

8.6 監査・証跡用途

最後に、最も地味で、最も重要な用途です。

監査や証跡において求められるのは、

- すべてを保存すること
- すべてを再現すること

ではありません。

必要なのは、

- 誰が
- いつ
- 何を承認したか

が、後から否定できない形で残っていることです。

署名は、その条件を自然に満たします。

- 改ざんされない
- 主体が明確
- 文脈を外に持てる

ブロックチェーンは、

証拠保管庫ではなく、署名の固定点として使われます。

8.7 保存する情報を増やさなくていい

本章で挙げた例は、特別なユースケースではありません。既存の業務を、そのまま署名に置き換えただけです。

重要なのは、

「これ以外にも何ができるか」ではなく、

これ以上、チェーンに保存する情報を
増やさなくていい

という設計判断です。

ユースケースは、今後増えていきます。

業務も、判断も、例外も増え続けるでしょう。

しかし、署名を主役に据えた瞬間、

ブロックチェーンに求められる役割は固定されます。

- 誰が

- いつ
- 何を承認したか

それ以上の情報を、
無理に背負わせる必要はありません。

署名ドリブン設計は、
**できることを増やす設計ではなく、
残すものを決める設計**です。

その結果、判断に付随する文脈や情報は、
システムではなく、判断した個人や組織に還元されます。

終章：Symbol は L0 として置くのが自然だ

ブロックチェーンは、しばしば「どのチェーンが正しいか」「どれが勝つか」といった文脈で語られます。

しかし本書で見てきたように、署名と承認を中心に据えて整理すると、チェーンを優劣で並べる問い自体が、設計上あまり意味を持たなくなります。

チェーンは信仰の対象ではなく、用途ごとに配置する部品です。

Symbol を L1 や L2 として捉えようと、どうしても他チェーンとの比較に引きずられます。一方で、

- 署名が確実に残る
- 承認者が特定できる
- 順序と時刻が安定して記録される

といった性質だけに注目すると、Symbol はアプリケーション層よりも、もう一段下に置いた方が整理しやすく見えてきます。

本書では、この配置を 便宜的に L0 (Layer 0) と呼ぶことにします。

L0 として置かれた Symbol は、アプリケーションを制御しません。業務ロジックにも踏み込みません。状態も最小限に留めます。記録するのは、「この時点で、この公開鍵が、これを承認した」という事実だけです。

ログイン、承認、同意、操作履歴。処理や表示はオフチェーンで完結してよい。けれど、最終的な署名の証跡だけは、あとから否定できない形で残っていてほしい。役割をここまで絞ると、Symbol の設計が過不足なく噛み合ってきます。

Symbol の上に載るものは、Web アプリでも構いません。既存の ID 基盤でも、他チェーンのスマートコントラクトでも構いません。

重要なのは、「すべてを Symbol に載せること」ではなく、「何を Symbol に委ね、何を委ねないか」を決めることです。

- 承認と署名は Symbol へ
- 処理と表示はオフチェーンへ

- ・ 価値移転や複雑なロジックは、別の得意なチェーンへ

そう整理すると、チェーン同士は競争相手ではなく、役割の異なる部品として並びます。

L0としての Symbol は、目立ちません。派手なユースケースを前面に出すこともないでしょう。

しかし、止まると困るが、普段は意識されない。そういう基盤は、長く使われます。

完璧な DID である必要はありません。包括的なプラットフォームである必要もありません。

- ・ 署名でログインできる仕組み
- ・ 承認履歴を検証できる仕組み
- ・ 「誰が OK したか」を組み込める部品

そうした小さな要素が積み重なることで、L0 としての価値は現実的に形成されていきます。

Symbol は、主役である必要はありません。しかし、主役が安心して立てる土台として考えると、ちょうどいい性質を持っています。

チェーンを上か下かで評価するのではなく、「どこに置くと無理がないか」で考える。本書では、その結果として、Symbol を L0 に置く、という整理をしました。

これは結論というより、一つの配置の提案にすぎません。

別の置き方を選ぶ人もいるでしょう。別のチェーンを選ぶ場面もあるでしょう。それで構いません。

重要なのは、ブロックチェーンを「信じる対象」にしないことです。使いすぎず、背負わせすぎず、役割を決めて置く。

署名を残し、それ以外を持たない。必要なところだけを使い、不要なところは手放す。

そういう設計ができるなら、チェーンの名前は、主役ではなくなります。

Symbol は、そのための一つの選択肢です。

付記

本書で書いたのは結論というより、ひとつの配置案です。設計はいつも、背負うものを減らした分だけ長く続きます。役割を決めて置く。その積み重ねが、現実にはいちばん強いのだと思っています。

注記

本書は生成 AI を補助的に利用して記述されています。ただし内容の責任は著者にあり、最終的な承認も著者が行っています。

署名済み（エクスプローラへのリンク：TODO）

奥付

書名: 使いすぎないブロックチェーン
副題: — Symbol で考える署名ドリブン設計 —
著者: さだかね
サークル: nemnesia
版: v1.0
発行日: 2026-02-15
権利表記: © 2026 さだかね
連絡先: X: @ccHarvestasya
if(repository) リポジトリ:
endif